



Bellman-Ford y Floyd-Warshall



Floyd Warshall

- **Problema:** Encontrar la longitud del camino más corto entre **cada par** de vértices en un grafo.
- El algoritmo utiliza **Programación Dinámica** para resolver el problema de forma iterativa.
- **Idea Central:** Un camino más corto de un nodo i a un nodo j solo puede ser:
 - El camino más corto conocido **anteriormente**.
 - Un nuevo camino que pasa por un **vértice intermedio** k
- **Restricción:** Solo es aplicable a grafos que **no contienen ciclos de peso negativo**.



Floyd Warshall

- Se define $D_k[i][j]$ como la distancia más corta de i a j utilizando solo vértices del conjunto $\{1, 2, \dots, k\}$ como intermedios.
- Para cada iteración k : $D_{\{k\}}[i][j] = \min(D_{\{k-1\}}[i][j], D_{\{k-1\}}[i][k] + D_{\{k-1\}}[k][j])$
- $D_{\{k-1\}}[i][j]$: Distancia más corta conocida sin usar k (Opción 1)
- $D_{\{k-1\}}[i][k] + D_{\{k-1\}}[k][j]$: La distancia más corta de i a k , más la de k a j (Opción 2, usando k)

Floyd Warshall



```
FloydWarshall( $W$ ):  
1    $D^{(0)} \leftarrow W$   
2   for  $k = 1 \dots n$  :  
3        $D^{(k)} \leftarrow (d_{ij}^{(k)})$  una nueva matriz  
4       for  $i = 1 \dots n$  :  
5           for  $j = 1 \dots n$  :  
6                $d_{ij}^{(k)} \leftarrow \min \left( d_{ij}^{(k-1)}, d_{ik}^{(k-1)} + d_{kj}^{(k-1)} \right)$   
7   return  $D^{(n)}$ 
```

Ejercicio 1

En clases se vio esta versión de Bellman-Ford, que encuentra el camino más corto (o “barato”) en un grafo direccional, acíclico.

Básicamente, el algoritmo hace $|V|-1$ pasadas por todas las aristas del grafo, tratando de reducirlas (o relajarlas). Sin embargo, las únicas aristas que podrían producir un cambio en $d[]$ son aquellas que salen de un vértice cuyo $d[]$ cambió en la pasada anterior.

```
1 BellmanFord(s):
2   for  $u \in V$  :
3        $d[u] \leftarrow \infty$ ;  $\pi[u] \leftarrow \emptyset$ 
4    $d[s] \leftarrow 0$ 
5   for  $k = 1 \dots |V| - 1$  :
6       for  $(u, v) \in E$  :
7           if  $d[v] > d[u] + \text{cost}(u, v)$  :
8                $d[v] \leftarrow d[u] + \text{cost}(u, v)$ 
9                $\pi[v] \leftarrow u$ 
```

Ejercicio 1

Modifica este algoritmo de manera que intente reducir aristas sólo cuando tenga sentido hacerlo.

HINT: Emplea una cola y un arreglo de booleanos que permitan detectar cuando hay que detenerse.

```
1 BellmanFord(s):
2   for u ∈ V :
3       d[u] ← ∞; π[u] ← ∅
4   d[s] ← 0
5   for k = 1 ... |V| - 1 :
6       for (u, v) ∈ E :
7           if d[v] > d[u] + cost(u, v) :
8               d[v] ← d[u] + cost(u, v)
9               π[v] ← u
```

Ejercicio 1



Nuevos elementos:

Cola **Q**: Donde iremos almacenando los nodos que han cambiado, y que son candidatos a alterar $d[]$

Arreglo de bools **Encolado**[] : Arreglo que nos dirá si el nodo v se encuentra en la cola Q

Ejercicio 1

Bellman-Ford'(s):

```
d[s] = 0                                     # suponemos que todos los otros d[] son inicialmente
∞
Q.enqueue(s)                                # suponemos que Q está inicialmente vacía
Encolado[s] = True                           # Todos los otros casilleros de Q son False, salvo s
while !Q.empty():                           # Iteramos mientras Q no esté vacía (y no |V| - 1 veces)
    u = Q.dequeue()
    Encolado[u] = False
    for each v in Vecinos[u]:                # Tratamos de reducir cada arista que sale de u
        if d[v] > d[u] +  $\omega(u,v)$ :
            d[v] = d[u] +  $\omega(u,v)$ 
             $\pi[v] = u$ 
            if !Encolado[v]:                 # si pudimos reducir (u,v), entonces
                Q.enqueue(v)                 # guardamos v en Q (si v no está ya en Q)
            Encolado[v] = True
```


Ejercicio 2



Dado un grafo $G = (V, E)$, con un vector de pesos w (todos positivos) y un nodo s de partida, Al profesor Luigi le interesa el problema de encontrar el costo del camino simple (sin ciclos) **de mayor costo entre s y cada nodo del grafo.**

Charles Walls, ex-alumno de IIC2133, argumenta que una modificación del algoritmo de Bellman-Ford (BF) puede resolver este problema. Su razonamiento es el siguiente:

“Como el camino que buscamos no tiene ciclos, el camino más largo entre dos nodos tiene a lo más $|V| - 1$ aristas. De esta forma podemos primero buscar los caminos más largos de 1 arista, luego los de dos aristas, y así sucesivamente, tal como lo hace BF.”

Ejercicio 2

El algoritmo propuesto por Charles es el siguiente:

```
1  BellmanFordn't(s):
2      for u ∈ V :
3          d[u] ← -∞; π[u] ← ∅
4      d[s] ← 0
5      for k = 1 ... |V| - 1 :
6          for (u, v) ∈ E :
7              if d[v] < d[u] + cost(u, v) :
8                  d[v] ← d[u] + cost(u, v)
9                  π[v] ← u
```

*Los únicos cambios con respecto a Bellman-Ford original son:

1. En lugar de ∞ se asigna $-\infty$
2. En lugar de $>$ tenemos un $<$.

Ejercicio 2

El profesor Luigi duda un poco de este algoritmo, por lo que te hace las siguientes preguntas:

- 1) Existe algún contraejemplo que muestre que este algoritmo no siempre funciona? Descríbelo
- 2) Debe cumplirse alguna condición para que este algoritmo funcione?Cuál es?
- 3) El profesor afirma que hay parte del argumento del alumno que efectivamente está correcto: **“Los caminos más largos no pueden tener más de $|V|-1$ aristas”**. Diga cómo es posible modificar la idea de BF para encontrar caminos más largos. Analice cuál sería la complejidad del algoritmo resultante. No es necesario que explicites el algoritmo.

```
1 BellmanFordn't(s):
2   for u ∈ V :
3       d[u] ← -∞; π[u] ← ∅
4   d[s] ← 0
5   for k = 1 ... |V| - 1 :
6       for (u, v) ∈ E :
7           if d[v] < d[u] + cost(u, v) :
8               d[v] ← d[u] + cost(u, v)
9               π[v] ← u
```

Ejercicio 2

- 
- 1) Existe algún contraejemplo que muestre que este algoritmo no siempre funciona?
Describe:

Respuesta: El contraejemplo se construye con un grafo con un ciclo en el camino hacia un nodo.

Ejercicio 2

- 
- 1) Existe algún contraejemplo que muestre que este algoritmo no siempre funciona?
Descríbelo:

Respuesta: El contraejemplo se construye con un grafo con un ciclo en el camino hacia un nodo.

- 1) Debe cumplirse alguna condición para que este algoritmo funcione?Cuál es?

Respuesta: Dado que la solución está basada en BF, podríamos pensar que encontrar el camino más caro sin ciclos en un grafo $G = (V, E)$ con pesos w es equivalente a encontrar el camino más barato sin ciclos en un grafo $G = (V, E)$ con $-w$. Como BF encontrará el camino más barato en un grafo sin ciclos negativos, debe cumplirse esta condición para que este nuevo algoritmo funcione

Ejercicio 2



3) El profesor afirma que hay parte del argumento del alumno que efectivamente está correcto: ***“Los caminos más largos no pueden tener más de $|V|-1$ aristas”***. Diga cómo es posible modificar la idea de BF para encontrar caminos más largos. Analice cuál sería la complejidad del algoritmo resultante. No es necesario que explicites el algoritmo.

Respuesta: Debemos modificar BF de forma que cada vez que se encuentra un nuevo camino hasta un nodo, se guarde explícitamente, junto a su costo (esto con el fin de poder ir actualizando cualquier solución más cara que encontremos).

Ejercicio 2



Esto significa que debemos tener una matriz $d[v][i]$ que almacena el costo del i -ésimo camino sin ciclos hasta v que se ha encontrado. El camino a su vez, se almacena en $\pi[v][i]$.

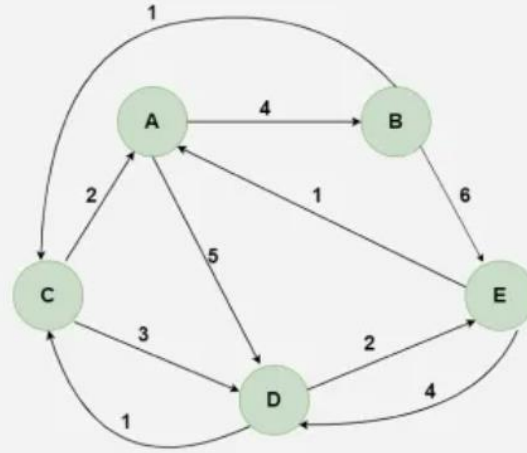
Cuando miramos (u, v) debemos usar esta arista completando cada camino hasta u para generar un nuevo camino hasta v . El costo de computar esto depende del tamaño de $d[v][i]$, que en el peor caso debe llevar la cuenta de todos los caminos posibles desde la fuente hasta v , que es exponencial en $|V|$ ($O(|V|!)$, de hecho)

El algoritmo principal, entonces, realiza $|V|-1$ iteraciones, pero cada iteración es $O(V !)$. El algoritmo es $O((|V| + 1)!)$.

Ejercicio 3

- Determina el camino más corto entre todos los pares de nodos i, j en el grafo

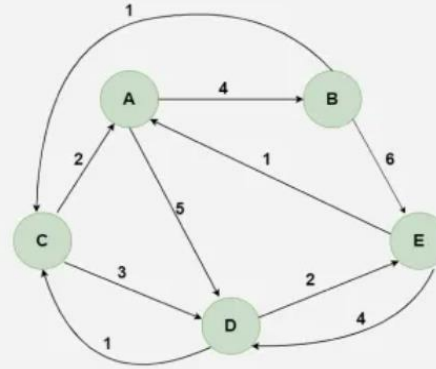
	A	B	C	D	E
A	0	4	10^8	5	10^8
B	10^8	0	1	10^8	6
C	2	10^8	0	3	10^8
D	10^8	10^8	1	0	2
E	1	10^8	10^8	4	0



Ejercicio 3

- Dada una matriz $\text{dist}[n][n]$ donde $\text{dist}[i][j]$ representa el peso desde el nodo i al nodo j (por ejemplo, $A \rightarrow B \rightarrow C$).
Si no existe un arco directo, entonces $\text{dist}[i][j] = \text{INF}$ (por ejemplo, 10^8) y, para todos los nodos, $\text{dist}[i][i] = 0$.

	A	B	C	D	E
A	0	4	10^8	5	10^8
B	10^8	0	1	10^8	6
C	2	10^8	0	3	10^8
D	10^8	10^8	1	0	2
E	1	10^8	10^8	4	0



Ejercicio 3

Tomamos el primer nodo (por ejemplo, A) como nodo intermedio y calculamos $\text{dist}[i][j]$ para cada par de nodos (i, j) usando la fórmula $\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][A] + \text{dist}[A][j])$.

	A	B	C	D	E
A	0	4	10^8	5	10^8
B	10^8	0	1	10^8	6
C	2	10^8	0	3	10^8
D	10^8	10^8	1	0	2
E	1	10^8	10^8	4	0

	A	B	C	D	E
A	0	4	10^8	5	10^8
B	10^8	0	1	10^8	6
C	2	6	0	3	10^8
D	10^8	10^8	1	0	2
E	1	5	10^8	4	0

Ejercicio 3

Tomamos el segundo nodo B como nodo intermedio y calcula $\text{dist}[i][j]$ para cada par de nodos (i, j) usando la fórmula

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][B] + \text{dist}[B][j]).$$

	A	B	C	D	E
A	0	4	10^8	5	10^8
B	10^8	0	1	10^8	6
C	2	6	0	3	10^8
D	10^8	10^8	1	0	2
E	1	5	10^8	4	0

	A	B	C	D	E
A	0	4	5	5	10
B	10^8	0	1	10^8	6
C	2	6	0	3	12
D	10^8	10^8	1	0	2
E	1	5	6	4	0

Ejercicio 3

Tomamos el tercer nodo C como nodo intermedio y calculamos $\text{dist}[i][j]$ para cada par de nodos (i, j) usando la fórmula

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][C] + \text{dist}[C][j]).$$

	A	B	C	D	E
A	0	4	5	5	10
B	10^8	0	1	10^8	6
C	2	6	0	3	12
D	10^8	10^8	1	0	2
E	1	5	6	4	0

	A	B	C	D	E
A	0	4	5	5	10
B	3	0	1	4	6
C	2	6	0	3	12
D	3	7	1	0	2
E	1	5	6	4	0

Ejercicio 3

Tomamos el 4to nodo D como nodo intermedio y calculamos $\text{dist}[i][j]$ para cada par de nodos (i, j) usando la fórmula

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][D] + \text{dist}[D][j]).$$

	A	B	C	D	E
A	0	4	5	5	10
B	3	0	1	4	6
C	2	6	0	3	12
D	3	7	1	0	2
E	1	5	6	4	0

	A	B	C	D	E
A	0	4	5	5	7
B	3	0	1	4	6
C	2	6	0	3	5
D	3	7	1	0	2
E	1	5	5	4	0

Ejercicio 3

Tomamos el 5to nodo E como nodo intermedio y calculamos $\text{dist}[i][j]$ para cada par de nodos (i, j) usando la fórmula

$$\text{dist}[i][j] = \min(\text{dist}[i][j], \text{dist}[i][E] + \text{dist}[E][j]).$$

	A	B	C	D	E
A	0	4	5	5	7
B	3	0	1	4	6
C	2	6	0	3	5
D	3	7	1	0	2
E	1	5	5	4	0

	A	B	C	D	E
A	0	4	5	5	7
B	3	0	1	4	6
C	2	6	0	3	5
D	3	7	1	0	2
E	1	5	5	4	0